

# UNIT I

## 1. Define Programming Language

A programming language is a language used to communicate with a computer and give instructions to perform tasks.

It contains:

- Keywords
- Symbols
- Rules (syntax)

**Example:** `printf("Hello");`

---

## 2. Explain Evolution of Programming Languages

Programming languages developed step by step to make programming easier and faster.

**Generations:**

1. Machine Language (0s and 1s)
  2. Assembly Language
  3. High Level Language (C, Java)
  4. Object Oriented Language
  5. AI Languages
- 

## 3. Define Attribute Grammar

Attribute grammar is an extension of CFG that includes additional information called attributes. It helps to describe semantics. **Example:** Type checking uses attribute grammar.

## 4. Define Syntax and Semantics

**Syntax** - Syntax means grammar or structure rules of a program.

**Example:** `int a = 10;`

**Semantics** - Semantics means meaning of the statement.

**Example :** `a = 10` means value 10 is stored in variable a.

---

## 5. Differentiate Sentence and Sentential Form

| Syntax               | Semantics              |
|----------------------|------------------------|
| Structure of program | Meaning of program     |
| Checks grammar rules | Checks logical meaning |
| Done during parsing  | Done during execution  |

---

## 6. Differentiate Syntax and Semantics

| Syntax               | Semantics              |
|----------------------|------------------------|
| Structure of program | Meaning of program     |
| Checks grammar rules | Checks logical meaning |
| Done during parsing  | Done during execution  |

## 7. Define Context Free Grammar (CFG)

Context Free Grammar is a set of rules used to describe the syntax of programming languages.

It contains:

- Non-terminals
- Terminals
- Productions
- Start symbol

### Example

$S \rightarrow aS$

$S \rightarrow b$

---

## 8. Write BNF notation for For Loop and If-Else

### For Loop

$\langle \text{for\_stmt} \rangle \rightarrow \text{for}(\text{initialization}; \text{condition}; \text{increment})$

### If-Else

$\langle \text{if\_stmt} \rangle \rightarrow \text{if}(\text{condition})$   
    statement  
    else  
    statement

---

## 9. Define Parse Tree

Parse tree is a tree representation of grammar derivation. It shows how a sentence is generated from grammar rules.

### Diagram

S  
/\nA B

## 10. Define Lexical Analysis

Lexical analysis is the first phase of the compiler. It converts the program into tokens.

### Example

```
int a = 5;
```

Tokens:

```
int | a | = | 5
```

---

## 11. Define Parsing

Parsing is the process of checking whether tokens follow grammar rules. It creates the parse tree.

---

## 12. Define Recursive Descent Parsing

Recursive descent parsing is a top-down parsing method. It uses recursive functions for each grammar rule.

**Example:** Each non-terminal has one function.

## 13. Define Bottom-Up Parsing

- Bottom-up parsing starts from input symbols and builds toward start symbol. It reduces small parts into larger parts.
  - Bottom-up parser works from bottom to top.
-

## 14. Define Denotational Semantics

Denotational semantics gives mathematical meaning to program statements. It maps program statements into mathematical functions.

---

## 15. Define Operational Semantics

Operational semantics explains program meaning by execution steps. It shows how statements execute one by one.

### Example

`a = a + 1;`

Value of `a` increases by 1.

---

## 16. Define Axiomatic Semantics

Axiomatic semantics is used to prove program correctness using logical rules.

It uses:

- Preconditions
- Postconditions

### Example

`{X > 0}`

`X = X + 1`

`{X > 1}`

---

## 17. Differentiate Compiler and Interpreter

| Compiler                         | Interpreter               |
|----------------------------------|---------------------------|
| Translates whole program at once | Translates line by line   |
| Faster execution                 | Slower execution          |
| Errors shown after compilation   | Errors shown line by line |

### Examples

- Compiler → C
  - Interpreter → Python
- 

## 18. Define Language Recognizers

Language recognizers are programs that check whether input belongs to a language.

**Example:** Parser is a language recognizer.

---

## 19. Define Generators

Generators are tools that automatically create language processors.

### Example

Parser generators:

- YACC
  - ANTLR
-

# UNIT II

## 1. Define Variable

A variable is a named memory location used to store data values.

### Example

```
int age = 20; // Here, age is a variable.
```

---

## 2. Define Binding

Binding is the process of connecting a variable with its type or value.

### Example

```
int a = 10;
```

---

## 3. Define Type Checking

Type checking checks whether data types are used correctly in a program.

### Example

```
int a = "Hello"; // Error
```

---

## 4. Define Scope and Scope Rules

**Scope:** Scope is the region where a variable can be accessed.

### Types

- Local scope
- Global scope

### Scope Rules

Rules that determine visibility of variables.

### **Example**

```
int x = 10;
```

---

## **5. Define Lifetime of a Variable**

Lifetime is the time during which a variable exists in memory.

**Example:** Local variable exists only inside function.

---

## **6. Define Garbage Collection**

Garbage collection is the process of removing unused memory automatically.

**Example Languages:** Java, Python

---

## **7. Define Primitive Data Types**

Primitive data types are basic built-in data types.

### **Examples**

- int
- char
- float
- boolean

### **Example Program**

```
int a = 5;  
char ch = 'A';
```

## **8. Define Strings**



A string is a collection of characters.

**Example:** `char name[] = "Technology";`

---

## 9. Define Arrays

An array is a collection of similar data elements stored in continuous memory locations.

**Example**

`int a[5];`

---

## 10. Define Associative Arrays

Associative arrays store data using keys and values.

**Example**

`student["name"] = "Ram"`

---

## 11. Define Record Type

A record type stores different types of data together.

**Example**

```
struct Student{  
    int id;  
    char name[20];  
};
```

## 12. Differentiate Union and Enumeration

| Union                      | Enumeration             |
|----------------------------|-------------------------|
| Shares same memory         | Defines named constants |
| Stores one value at a time | Stores fixed values     |

### Example

```
union data{
    int a;
    float b;
};
enum day{Mon,Tue,Wed};
```

---

## 13. Define Pointers and References

**Pointer** - Pointer stores the address of another variable.

**Example** : `int *p;`

**Reference** - Reference is another name for a variable.

**Example:** `int &x = a;`

---

## 14. Define Widening and Narrowing Conversions

**Widening Conversion** - Converting smaller types into larger types.

**Example** `int a = 10; float b = a;`

**Narrowing Conversion**- Converting larger types into smaller types.

**Example:** `float x = 10.5; int y = x;`

---

## 15. Define Arithmetic Expressions

Arithmetic expressions perform mathematical operations.

**Operators:** + - \* / %

**Example:** `c = a + b;`

---

## 16. Define Overloaded Operators

Operator overloading means using the same operator for different purposes.

**Example:** + (Plus Operator)

- Addition of numbers
  - Joining strings
- 

## 17. Define Type Conversion

Type conversion changes one data type into another.

**Example**

`float x = (float)10;`

---

## 18. Define Relational Expressions

Relational expressions compare two values.

**Operators:** < > <= >= != ==

**Example:** a > b

---

## 19. Define Boolean Expressions

Boolean expressions produce Boolean values:

- True
- False

### Operators

- AND (&&)
- OR (||)
- NOT (!)

### Example

(a > b) && (b > c)

---

## 20. Define Assignment Statement

Assignment statement stores value into variable.

### Example

a = 10;

## 21. Define Mixed Mode Assignment

Mixed mode assignment uses different data types in the same expression.

### Example

```
int a = 5;  
float b = 2.5;  
float c = a + b;
```

---

## 22. Define Control Structures

Control structures control flow of program execution.

### Types

- Selection
  - Iteration
  - Branching
- 

## 23. Define Iteration Statements

Iteration statements repeat a block of code.

### Examples

- for
- while
- do while

**Example:** `for(i=0;i<5;i++)`

---

## 23. Define Selection Statements

Selection statements choose one path among many paths.

**Example:** `if(a>b)`

---

## 25. Define Branching Statements

Branching statements transfer control to another statement.

### Examples

- break
- continue
- goto

## UNIT III

### 1. Define Subprogram

A subprogram is a group of statements that performs a specific task.

It can be reused many times in a program.

## Types

- Function
  - Procedure
- 

## 2. Define Procedures

A procedure is a subprogram that performs an action but does not return a value.

### Example

```
void display()
{
    printf("Hello");
}
```

---

## 3. Define Functions

A function is a subprogram that returns a value after execution.

### Example

```
int square(int x)
{
    return x*x;
}
```

## 4. List Characteristics of Subprograms

Characteristics of subprograms:

1. Reusability

2. Modularity
  3. Reduced code size
  4. Easy debugging
  5. Better readability
- 

## 5. Define Local Referencing Environment

The local referencing environment contains all variables visible inside a subprogram.

### Example

```
void fun()
{
    int a = 10;
}
```

Here **a** belongs to the local environment.

---

## 6. Define Parameter Passing

Parameter passing is the process of sending values to a subprogram.

### Types

- Pass by value
- Pass by reference

**Example:** `sum(a,b);`

## 7. Define Pass by Value

In pass by value, a copy of the actual value is sent to the function. The original value will not change.



### Example

```
void fun(int x)
{
    x = 20;
}
```

---

## 8. Define Pass by Reference

In pass by reference, the address of the variable is sent to the function. Original value can be modified.

### Example

```
void fun(int *x)
{
    *x = 20;
}
```

---

## 9. Define Overloaded Methods

Overloaded methods have the same name but different parameters.

### Example

```
add(int a,int b)
add(float a,float b)
```

---

## 10. Define Pass by Result

In pass by result, value is returned through parameter after function execution.

---

## 11. Define Generic Methods

Generic methods work with different data types using one method.

### Example

```
template<class T>  
T add(T a,T b)
```

---

## 12. Define Semantics of Call and Return

It explains how control transfers during function call and return.

### Steps

1. Function call occurs
  2. Control transfers to function
  3. Function executes
  4. Control returns back
- 

## 13. Define Stack Dynamic Local Variables

Variables created during function execution and stored in stack memory are called stack dynamic local variables.

### Example

```
void fun()  
{  
    int a;  
}
```

---

## 14. Define Dynamic Local Variables

Dynamic local variables are created during runtime. Memory is allocated dynamically.

**Example:** `int *p = malloc(sizeof(int));`

---

## 15. Define Nested Subprograms

A subprogram defined inside another subprogram is called a nested subprogram.

### Example

Function A

└─ Function B

---

## 16. Define Blocks

A block is a group of statements enclosed within braces `{ }`.

### Example

```
{  
  int a = 10;  
}
```

## 17. Define Static Scope

In static scope, variable reference depends on program structure. It is determined during compile time.

## Example

Variable searched in the nearest enclosing block.

---

## 18. Define Dynamic Scope

In dynamic scope, variable reference depends on function calling sequence.

It is determined during runtime.

---

## 19. Differentiate Static and Dynamic Binding

| Static Binding                | Dynamic Binding               |
|-------------------------------|-------------------------------|
| Binding done before execution | Binding done during execution |
| Faster                        | Slower                        |
| Compile time                  | Runtime                       |

---

## 20. Differentiate Shallow and Deep Binding

| Shallow Binding          | Deep Binding              |
|--------------------------|---------------------------|
| Uses current environment | Uses old environment      |
| Binding during call      | Binding during definition |

# UNIT IV

## 1. Define Object Oriented Programming (OOP)

Object Oriented Programming is a programming method based on objects and classes.

## Four Pillars of OOP

- Encapsulation
  - Inheritance
  - Polymorphism
  - Abstraction
- 

## 2. Define Encapsulation

Encapsulation means binding data and methods together into one unit. It also hides data from outside access.

### Example

```
class Account
{
    private int balance;
}
```

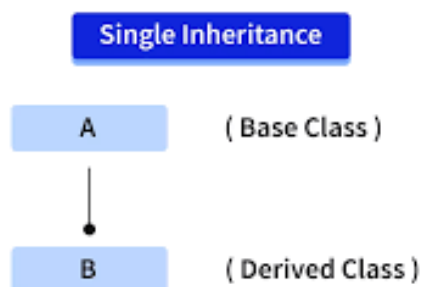
---

## 3. Define Inheritance

Inheritance allows one class to acquire properties of another class.

### Example

```
class A{}
class B extends A{}
```



## 4. Define Polymorphism

Polymorphism means one function or operator behaves differently in different situations.

### Example

```
add(int,int)
add(float,float)
```

---

## 5. Define Abstraction

Abstraction means showing important features and hiding unnecessary details.

**Example:** ATM machines hide internal processes.

---

## 6. Define Concurrency

Concurrency means executing multiple tasks at the same time.

**Example:** Downloading file while listening to music.

---

## 7. Define Semaphore

Semaphore is a synchronization tool used to control access to shared resources.

### Types

- Binary semaphore
  - Counting semaphore
- 

## 8. Define Binary Semaphore

Binary semaphore can have only two values:

- 0

- 1

It is used for mutual exclusion.

---

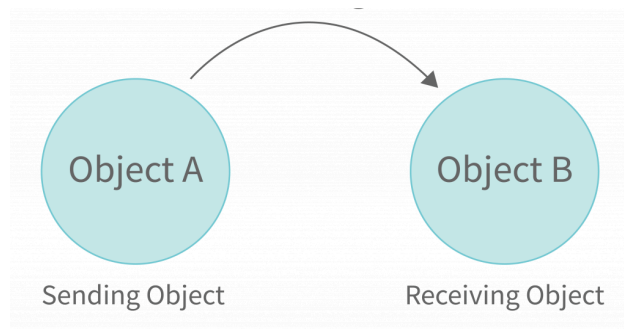
## 9. Define Monitor

Monitor is a synchronization construct used to allow safe access to shared data. Only one process can use a monitor at a time.

---

## 10. Define Message Passing

Message passing is a communication method where processes exchange messages.



## 11. Define Thread

A thread is a small unit of execution inside a process.

**Example:** Browser tabs running simultaneously.

---

## 12. Define Statement Level Concurrency

Statement level concurrency means multiple program statements execute simultaneously.

---

### 13. Define Mutual Exclusion

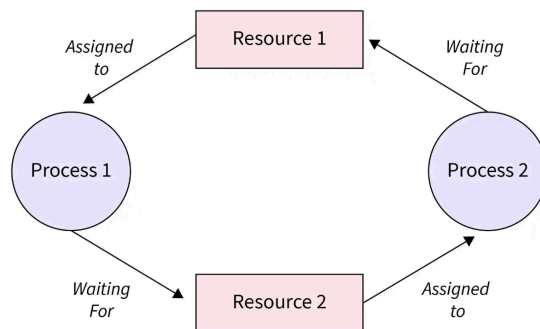
Mutual exclusion ensures only one process accesses shared resources at a time.

**Example:** One printer used by one user at a time.

---

### 14. Define Deadlock

Deadlock occurs when two or more processes wait forever for resources held by each other.



### 15. Define Exception Handling

Exception handling is a method of handling runtime errors.

**Example**

```
try {}  
catch(Exception e)  
{}
```

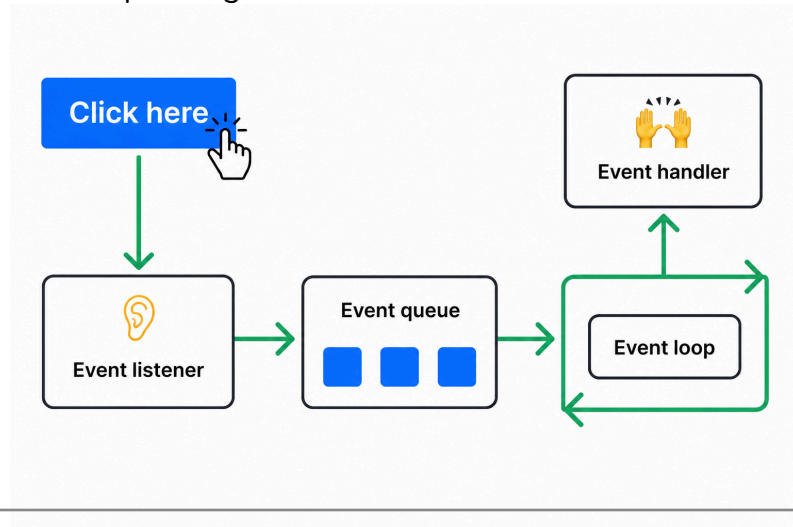


## 16. Define Event Handling

Event handling is the process of responding to user actions.

### Examples

- Mouse click
- Keyboard press
- Button click



## 17. Define Abstract Data Type (ADT)

ADT is a data type where only operations are visible and implementation is hidden.

### Example

Stack

- Push
- Pop

## 18. Define Data Abstraction

Data abstraction hides implementation details and shows only essential information.

### Example

Car driving without knowing engine details.

## 19. Define Logic Programming Language

Logic programming language uses facts and rules to solve problems.

**Example:** Prolog

---

## 20. Define Rule and Goal Statements in Prolog

**Rule Statement** - Defines relationship between facts.

**Example**

father(X,Y):- parent(X,Y), male(X).

**Goal Statement** - Query used to get results.

**Example**

?- father(ram,ravi).

# UNIT V

## 1. Define Lambda Calculus

Lambda calculus is a mathematical system used as the foundation of functional programming languages.

It uses functions for computation.

**Example:**  $\lambda x.x+1$

---

## 2. Define Functional Programming Language

A functional programming language uses functions as the main building blocks of programs. It avoids changing data repeatedly.

### Examples

- Lisp
  - Scheme
  - ML
  - Haskell
- 

## 3. Define Procedural Abstraction

Procedural abstraction hides implementation details of procedures or functions.

User only knows:

- Function name
- Inputs
- Outputs

## 4. Define Referential Transparency

Referential transparency means expression always produces the same result for the same input.

### Example

$5 + 3 = 8$  //Always gives the same result.

---

## 5. Define Lazy Evaluation

Lazy evaluation delays computation until a result is needed.

**Advantage:** Saves memory and time.

---

## 6. Define Imperative Language

Imperative language focuses on step-by-step instructions to solve problems.

### Examples:

- C
  - Pascal
- 

## 7. Define Scripting Language

Scripting language is used to automate tasks and control applications.

### Examples

- Python
- JavaScript
- Perl

## 8. Define Scheme Programming Language

Scheme is a functional programming language derived from Lisp. It uses simple syntax and recursion.

**Example:** (+ 2 3)

---

## 9. Define ML Programming Language

ML (Meta Language) is a functional programming language with strong type checking.

### Features

- Functional programming
  - Type safety
  - Pattern matching
- 

## 10. Define Prolog

Prolog is a logic programming language based on facts and rules.

**Example:** father(ram,ravi).

---

## 11. Define Logic Programming

Logic programming is a programming method based on facts, rules and logical reasoning.

### Example

Prolog language.

---

## 12. Define Fact Statement in Prolog

The fact statement represents true information in Prolog.

**Example:** `student(ram).`

---

## 13. Define List Comprehension in Haskell

List comprehension is a simple method to create lists in Haskell.

**Example**

```
[x*x | x <- [1..5]]
```

**Output**

```
1 4 9 16 25
```

---

## 14. Define Python Variables

The Python variable stores data values.

**Example:** `x = 10`

---

## 15. Define Scope in Python

Scope determines where a variable can be accessed.

**Types**

- Local scope
- Global scope

**Example:** `x = 10`

## 17. List Characteristics of Python Language

Characteristics:

- Simple
  - Easy to learn
  - Portable
  - Interpreted
  - Object oriented
- 

## 18. List Data Types in Python

Python data types:

- int
  - float
  - string
  - list
  - tuple
  - dictionary
- 

## 19. List Examples of Scripting Languages

Examples:

- Python
- JavaScript
- PHP
- Perl
- Ruby